

Coding & Solution

Date	04 October 2026
Team ID	POCKETSMART-TEAM
Project Name	PocketSmart AI – Smart Budget & Recommendation Assistant using Gemini Models
Maximum Marks	3 Marks

Core Implementation

1. Home, Party and Jewelry Recommendations

`get_home_recommendations()`, `get_party_recommendations()` and `get_jewelry_recommendations()` in `gemini_utils.py` build a prompt from the validated input, call Gemini, parse the JSON reply and add shopping links. The jewelry function also accepts an outfit image and sends it to Gemini with the prompt.

2. Gemini Call with Model Fallback and Retry

`_generate_with_retry()` tries `gemini-3.5-flash-lite`, `gemini-2.5-flash-lite` and `gemini-flash-lite-latest` in order. A `NotFound` error moves to the next model immediately; any other error is retried once on the same model before moving on.

3. JSON Extraction

`extract_json_from_response()` removes markdown code fences and extracts the outermost JSON object, raising a clear error if none is found.

4. Static Fallback

`_home_fallback()`, `_party_fallback()` and `_jewelry_fallback()` return a rule-based percentage split of the budget, marked `source = "fallback"`, so the page can show a notice.

5. Authentication and Sessions

`auth.py` hashes passwords with `bcrypt`, issues `HS256` JWTs valid for 30 minutes, blacklists tokens on sign-out or re-login, and tracks sessions. `main.py` removes sessions idle for more than 30 minutes using a background task.

6. Validation and Uploads

`models.py` uses Pydantic validators (positive budgets, non-negative counts, at least one guest, non-empty text). `save_upload_file()` accepts only `.jpg`, `.jpeg`, `.png` and `.webp` files up to 5 MB and sanitises the file name.

7. History

`save_to_history()` stores each result with a UUID and timestamp; `/recommendation-history` lists them and `/recommendation-details/{id}` returns the full result.

Team Contribution

Member	Main Coding Contribution
Backend Lead	FastAPI routing, authentication, sessions and integration
Frontend Lead	HTML/Jinja2 templates, CSS and JavaScript helpers

AI Integration Lead	Gemini prompts, model fallback, JSON parsing and shopping links
Testing & Documentation Lead	Pydantic validation, fallback estimates, tests and documentation